

relevant-when: Open this drawer when creating formatted Excel workbooks with openpyxl and pandas DataFrames.

openpyxl Excel Reporting

Use openpyxl for structure, styles, layout, filters, and freeze panes. Use pandas only to supply tabular data.

Workbook and Multi-Sheet Creation

Create the workbook once, remove the default sheet, and keep sheet names valid and short.

```
from pathlib import Path

import pandas as pd
from openpyxl import Workbook
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
from openpyxl.utils import get_column_letter
from openpyxl.utils.dataframe import dataframe_to_rows

def safe_sheet_name(name: str) -> str:
    forbidden = str.maketrans({char: "_" for char in r'[]:*/\\'})
    return name.translate(forbidden)[:31] or "Sheet"

def create_report(sheets: dict[str, pd.DataFrame], output_path: Path) -> None:
    wb = Workbook()
    wb.remove(wb.active)
    for title, df in sheets.items():
        ws = wb.create_sheet(safe_sheet_name(title))
        write_dataframe(ws, df, start_row=3, start_col=1)
        format_report_sheet(ws, title)
    wb.save(output_path)
```

Writing DataFrames into Sheets

dataframe_to_rows() writes headers and values without carrying pandas index metadata unless requested.

```
def write_dataframe(ws, df: pd.DataFrame, start_row: int = 1, start_col: int = 1) -> None:
    for row_offset, row in enumerate(dataframe_to_rows(df, index=False, header=True)):
        for col_offset, value in enumerate(row):
            ws.cell(row=start_row + row_offset, column=start_col + col_offset, value=value)
```

Fonts, Borders, Fills, Alignment

Define reusable styles once. Assign style objects during sheet formatting.

```
HEADER_FONT = Font(name="Calibri", size=11, bold=True, color="FFFFFF")
TITLE_FONT = Font(name="Calibri", size=14, bold=True, color="1F4E79")
HEADER_FILL = PatternFill("solid", fgColor="1F4E79")
ALT_FILL = PatternFill("solid", fgColor="F2F6FA")
THIN_SIDE = Side(style="thin", color="D9E2F3")
THIN_BORDER = Border(left=THIN_SIDE, right=THIN_SIDE, top=THIN_SIDE, bottom=THIN_SIDE)
CENTER = Alignment(horizontal="center", vertical="center")
LEFT = Alignment(horizontal="left", vertical="center")
RIGHT = Alignment(horizontal="right", vertical="center")
```

Apply styles by row role.

```
def style_header_row(ws, header_row: int) -> None:
    for cell in ws[header_row]:
```

```

cell.font = HEADER_FONT
cell.fill = HEADER_FILL
cell.border = THIN_BORDER
cell.alignment = CENTER

```

```

def style_data_region(ws, first_data_row: int) -> None:
    for row in ws.iter_rows(min_row=first_data_row, max_row=ws.max_row):
        for cell in row:
            cell.border = THIN_BORDER
            cell.alignment = RIGHT if isinstance(cell.value, (int, float)) else LEFT
        if row[0].row % 2 == 0:
            for cell in row:
                cell.fill = ALT_FILL

```

Number Formats

Map formats from stable column names. Keep formatting separate from conversion.

```

NUMBER_FORMATS = {
    "amount": '#,##0.00',
    "balance": '#,##0.00',
    "pct": '0.00%',
    "date": 'yyyy-mm-dd',
}

```

```

def apply_number_formats(ws, header_row: int, first_data_row: int) -> None:
    headers = {ws.cell(header_row, col).value: col for col in range(1, ws.max_column + 1)}
    for header, col in headers.items():
        header_text = str(header or "").lower()
        matched = next((fmt for key, fmt in NUMBER_FORMATS.items() if key in header_text), None)
        if matched:
            for row in range(first_data_row, ws.max_row + 1):
                ws.cell(row, col).number_format = matched

```

Auto-Fit Columns

openpyxl cannot ask Excel to auto-fit. Calculate width from visible values.

```

def autofit_columns(ws, min_width: int = 10, max_width: int = 45) -> None:
    for col_idx in range(1, ws.max_column + 1):
        letter = get_column_letter(col_idx)
        longest = 0
        for cell in ws[letter]:
            if cell.value is not None:
                longest = max(longest, len(str(cell.value)))
        ws.column_dimensions[letter].width = min(max(longest + 2, min_width), max_width)

```

Freeze Panes and Filters

Set freeze panes to the first scrollable cell. Set the filter reference to the header row through the last data row.

```

def apply_navigation(ws, header_row: int) -> None:
    last_col = get_column_letter(ws.max_column)
    ws.freeze_panes = ws.cell(row=header_row + 1, column=1).coordinate
    ws.auto_filter.ref = f"A{header_row}:{last_col}{ws.max_row}"

```

Complete Formatter

```
def format_report_sheet(ws, title: str, header_row: int = 3) -> None:
    ws["A1"] = title
    ws["A1"].font = TITLE_FONT

    style_header_row(ws, header_row)
    style_data_region(ws, header_row + 1)
    apply_number_formats(ws, header_row, header_row + 1)
    apply_navigation(ws, header_row)
    autofit_columns(ws)
```